

Отчёт по лабораторной работе № 5

Название работы: Модель ветвления Git Flow

Студент: Пашовкин Артём Алексеевич

Группа: AT-13

Дата выполнения: 16.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Инициализировать репозиторий, создать ветку develop от main	☒	<pre>mkdir lab5_gitflow && cd lab5_gitflow git init echo "# Git Flow" > README.md git add . git commit -m "Initial main" git checkout -b develop [master (root-commit) 923519e] Initial main 1 file changed, 1 insertion(+) create mode 100644 README.md Switched to a new branch 'develop'</pre>
2	Создать feature-ветку login от develop, добавить auth.cfg, смержить с --no-ff	☒	<pre>git checkout -b feature/login develop echo "login: admin" > auth.cfg git add auth.cfg git commit -m "Add auth config" git checkout develop git merge --no-ff feature/login</pre>

Шаг	Описание	Вып.	Команды / вывод
			-m "Merge feature/login" Switched to a new branch 'feature/login' [feature/login cf2ade0] Add auth config 1 file changed, 1 insertion(+) create mode 100644 auth.cfg Switched to branch 'develop' Merge made by the 'ort' strategy. auth.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 auth.cfg
3	Создать feature-ветку db от develop, добавить db.cfg, смиржить с --no-ff	☒	git checkout -b feature/db develop echo "db_host=localhost" > db.cfg git add . git commit -m "Add DB config" git checkout develop git merge --no-ff feature/db - m "Merge DB feature" Switched to a new branch 'feature/db' [feature/db 8f732fc] Add DB config 1 file changed, 1 insertion(+) create mode 100644 db.cfg Switched to branch 'develop' Merge made by the 'ort' strategy. db.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 db.cfg
4	Создать release-ветку v1.0 от develop, добавить version.txt, сделать КОММИТ	☒	git checkout -b release/v1.0 develop echo "version=1.0" > version.txt

Шаг	Описание	Вып.	Команды / вывод
			git add version.txt git commit -m "Bump to 1.0" Switched to a new branch 'release/v1.0' [release/v1.0 b330bc5] Bump to 1.0 1 file changed, 1 insertion(+) create mode 100644 version.txt
5	Слить release в main (с --no-ff), создать тег v1.0	☒	git checkout main git merge --no-ff release/v1.0 -m "Release v1.0" git tag -a v1.0 -m "Version 1.0" Switched to branch 'main' Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt
6	Слить release обратно в develop (back-merge) с --no-ff	☒	git checkout develop git merge --no-ff release/v1.0 -m "Back-merge release" Switched to branch 'develop' Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt
7	Создать hotfix-ветку critical от main, добавить patch.log, сделать коммит	☒	git checkout main git checkout -b hotfix/critical main echo "patch applied" > patch.log git add . git commit -m "Critical security fix"

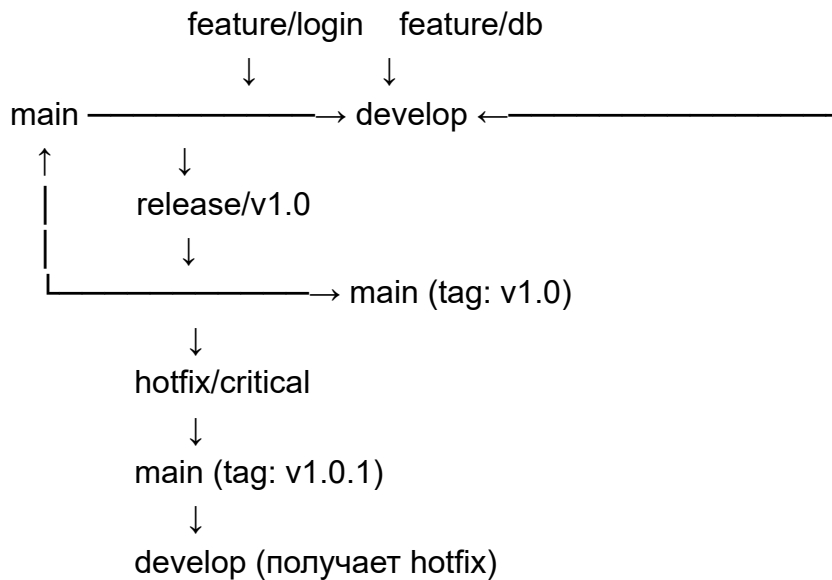
Шаг	Описание	Вып.	Команды / вывод
			Switched to branch 'main' Switched to a new branch 'hotfix/critical' [hotfix/critical 83d02df] Critical security fix 1 file changed, 1 insertion(+) create mode 100644 patch.log
8	Слить hotfix в main (с --no-ff), создать тег v1.0.1	☒	git checkout main git merge --no-ff hotfix/critical -m "Hotfix critical" git tag v1.0.1 Switched to branch 'main' Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log
9	Слить hotfix в develop (с --no-ff)	☒	git checkout develop git merge --no-ff hotfix/critical -m "Merge hotfix into develop" Switched to branch 'develop' Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log

2. Самостоятельное задание

Условие:

Нарисуйте схему Git Flow. Объясните, почему release сливается и в main, и в develop.

Выполнение (ход действий и результат):



3. Ответы на контрольные вопросы

1. Вопрос 1:

--no-ff (no fast-forward) создаёт отдельный коммит слияния

зачем нужен:

1. Сохраняется история в виде дерева
2. Легко откатить целую фичу одним коммитом
3. Понятно, какие коммиты относятся к какой фиче

2. Вопрос 2:

в теории можно, но нельзя по правилам git flow

почему это плохая практика:

1. **develop** — интеграционная ветка, туда должны попадать только проверенные фичи через merge
2. невозможно отследить, какая фича что добавила
3. если фича сломает сборку, сложно найти ее
4. теряется смысл модели git flow

3. Вопрос 3:

выдает критическую ошибку

что произойдёт:

1. исправление будет только в **main** (продакшене)
2. в **develop** исправления НЕТ
3. при следующем релизе (из **develop**) старая ошибка **ВЕРНЁТСЯ** в продакшен
4. придётся снова создавать hotfix

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.